

Python for Security Tooling — RE, VR and analysis

Concept cards — generated 2026-08-06

Contents

Track P	1
Module P01-tool-shape	1
The shape of a security tool	1
The import landscape, and the three interpreters	7
Module P02-bytes-and-struct	11
Bytes, struct, and the layout you were handed	11
Module P03-files-and-streams	16
Files you cannot fit in memory, archives you cannot trust	16
Module P04-containers	20
Parsing a container you did not write	20
Module P07-networking	26
recv does not return a message	26
Module P08-static-analysis	31
Passes: scanning five thousand functions	31

Track P

Module P01-tool-shape

The shape of a security tool

Every tool in this course has the same shape, and it is worth fixing that shape now because you are going to build eleven more of them.

A security tool is a library function that returns data, with a CLI bolted on that prints it.

The two halves fail differently, are tested differently, and — on this platform — are graded differently. The library half has a right answer, so the trainer grades it. The CLI half is yours: you assemble it from the template at the bottom of this card, and it goes in your repository.

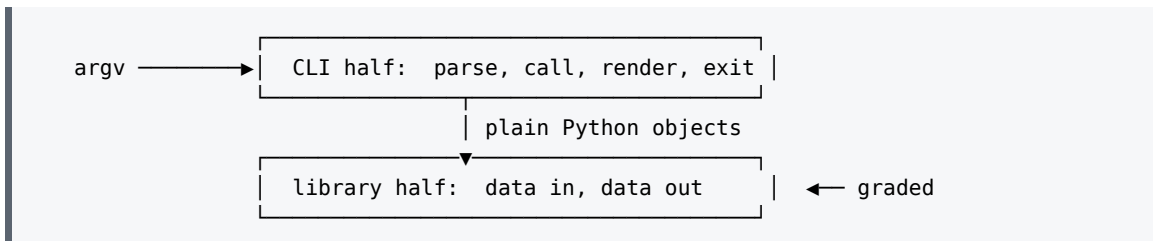
Why the split is not just tidiness

A tool written as one function that parses `sys.argv`, does the work, and prints as it goes cannot be:

- **called by another tool** — and the whole of P12 is composing these eleven tools into a pipeline;
- **tested** — you can test a return value; testing printed output means capturing stdout and matching strings, which breaks every time you improve a message;

- **reused at a different scale** — the function that analyses one file is the function you map over forty thousand.

So: the library half takes data and returns data. It does not print, it does not read `sys.argv`, and it does not call `sys.exit`. The CLI half does all three and nothing else.



Two streams, and they are not interchangeable Data goes to stdout. Everything else goes to stderr.

This is not style. It is what makes a tool composable:

```

binid --json ./firmware/* > findings.json    # data captured
                                           # progress still on your terminal
  
```

If your progress messages went to stdout, `findings.json` is now unparseable and you will not find out until something downstream chokes on it. The rule generalises: **if a human reads it, stderr; if a program reads it, stdout.**

A corollary that catches people: `print()` writes to stdout. In a tool, `print` is for results only. Progress, warnings and errors go through `logging`, which writes to stderr by default.

The exit-code contract

A tool that always exits 0 is telling every script that calls it that everything was fine. Frequently it was not.

The contract this course uses:

Exit	Meaning
0	The run completed and found nothing.
1	The run completed and found something worth a human's attention.
2	The run did not complete. Something failed to analyse.

Three things about it are worth stating explicitly, because they are where the mistakes live.

“Found nothing” and “could not look” are different answers. This is the one that matters. A scanner that fails to parse every single input and reports a clean run has not given you a clean

run; it has given you no run at all, dressed as a clean one. If anything errored, the exit code says so — even if the parts that worked found nothing.

Findings from a target that errored do not count. If your tool collected two findings from a file and then hit a malformed header and gave up on it, you do not know whether those findings are real, and you certainly do not know what else was in there. Discard them and report the error. A partial analysis reported as a complete one is the worst output a tool can produce, because it looks exactly like a good one.

Errors and findings can both be true at once. Forty files, thirty-six clean, four unparseable, and one finding among the thirty-six. That run found something *and* failed. Report both; the exit code takes the finding branch, because a finding is the thing a human must act on, and the error count in the report is what tells them the picture is incomplete.

This particular numbering is a convention, not a standard. `sysexits.h` and POSIX only agree that zero means success. What matters is that your tool has *a* contract, that it is documented in your README, and that it distinguishes “nothing found” from “could not look”.

Determinism, and why a reviewer cares

Run your tool twice on the same input. If the output differs, nobody can review it, diff it, or trust it.

The four usual sources:

- **Filesystem order.** `os.walk` and `Path.rglob` return entries in whatever order the filesystem gives them, which differs between machines and between runs. Sort before you emit.
- **Set and dict iteration.** Dicts preserve insertion order in modern Python; sets do not preserve anything. Anything that reaches your output through a set needs an explicit sort with an explicit tie-break.
- **Unseeded randomness.** Covered properly in P11, where a fuzzer you cannot replay is a fuzzer you cannot debug. The same rule applies to any sampling.
- **Timestamps and paths in the output body.** Put them in a provenance block, not scattered through the findings, so two runs diff cleanly.

The result record

Give a finding a shape and use the same one everywhere. A `dataclass` costs three lines and buys you type checking, a free `__repr__`, and one place to change when the shape grows:

```
from dataclasses import dataclass, asdict, field

@dataclass(frozen=True, slots=True)
class Finding:
    id: str  # stable, greppable: "unbounded-copy"
```

```

severity: str          # high | medium | low
target: str            # what it was found in
detail: str = ""       # one line, human-readable
evidence: dict = field(default_factory=dict)

def as_dict(self) -> dict:
    return asdict(self)

```

frozen=True because a finding should not be edited after it is made, and slots=True because you may end up with a great many of them.

The CLI wrapper — **toolcore**

This is the half you assemble. Paste it, keep it, and import it from every tool you build in this course. It is complete and runnable as it stands.

```

"""toolcore – the CLI and report spine shared by every tool in this course."""

from __future__ import annotations

import argparse
import json
import logging
import sys

log = logging.getLogger("tool")

EXIT_CLEAN, EXIT_FINDINGS, EXIT_ERROR = 0, 1, 2

def build_parser(prog: str, description: str) -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog=prog, description=description)
    p.add_argument("--json", action="store_true", help="emit JSON on stdout")
    p.add_argument("-v", "--verbose", action="count", default=0,
                    help="-v for info, -vv for debug")
    p.add_argument("--quiet", action="store_true", help="errors only")
    return p

def configure_logging(args: argparse.Namespace) -> None:
    """Exactly one handler, on stderr. Applications configure; libraries do not."""
    level = logging.WARNING
    if args.quiet:
        level = logging.ERROR
    elif args.verbose >= 2:
        level = logging.DEBUG
    elif args.verbose == 1:
        level = logging.INFO
    logging.basicConfig(
        level=level,
        stream=sys.stderr,
        format="%(levelname)-7s %(name)s: %(message)s",

```

```

)

def emit(report: dict, *, as_json: bool) -> None:
    """Data to stdout. Sorted keys so two runs diff cleanly."""
    if as_json:
        json.dump(report, sys.stdout, indent=2, sort_keys=True, default=str)
        sys.stdout.write("\n")
        return
    for line in render_text(report):
        print(line)

def render_text(report: dict) -> list[str]:
    lines = [f"{report['targets']} target(s): "
             f"{report['analysed']} analysed, {report['errors']} errored"]
    for f in report.get("findings_list", []):
        lines.append(f" [{f['severity']:<6}] {f['target']}: {f['id']}")
    lines.append(f"status: {report['status']}")
    return lines

def main(argv: list[str] | None = None) -> int:
    parser = build_parser("tool", "one-line description of your tool")
    parser.add_argument("paths", nargs="+", help="what to analyse")
    args = parser.parse_args(argv)
    configure_logging(args)

    results = []
    for path in sorted(args.paths):          # sorted: determinism
        try:
            results.append(analyse(path))    # <-- your library half
        except Exception as exc:            # noqa: BLE001 – recorded, not swallowed
            log.error("%s: %s", path, exc)
            results.append({"target": path, "findings": [], "error": str(exc)})

    report = summarize(results)              # <-- the graded function
    emit(report, as_json=args.json)
    return report["exit_code"]

if __name__ == "__main__":
    sys.exit(main())

```

Note the `except` block. It catches broadly *and records what it caught* as a result with an `error` set. That is the difference between handling an error and hiding one: the run continues, and the report still knows the picture is incomplete. `py-p1-hunt-silent-success` is the version of this loop that gets it wrong.

pyproject.toml

```
[project]
name = "toolcore"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = []

[project.scripts]
tool = "toolcore:main"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
```

`pip install -e .` and `tool` is on your PATH. The `[project.scripts]` entry point is what makes it a tool rather than a script you have to remember the path to.

README.md

```
# toolname

One sentence: what it finds, and in what.

## Install
    pip install -e .

## Use
    toolname [--json] [-v] PATH [PATH ...]

## Exit codes
| 0 | completed, nothing found |
| 1 | completed, findings reported |
| 2 | did not complete – see stderr |

## Example
    $ toolname --json ./samples/
    { "targets": 3, "analysed": 3, "errors": 0, ... }

## What it does not do
Be honest here. A tool whose limits are documented is more useful than one
that appears to do everything.
```

That last section is not filler. Every tool in this course narrows the field rather than deciding — a scanner reports candidates, not bugs. Writing the limit down is what separates a tool a reviewer can use from one they have to second-guess.

What is graded

You assemble the wrapper above; nothing checks it, and it is yours.

What the trainer grades is the function the wrapper calls on its second-to-last line:

```
summarize(results) -> dict
```

Given one result record per target, it folds them into a report with an honest status and the exit code this card's contract requires. That is `py-pl-summarize-code`, and it is the load-bearing half — everything above it is plumbing you paste once and reuse eleven times.

The import landscape, and the three interpreters

Most of the skill in this course is knowing what to reach for. The libraries are well documented individually and nowhere collected, so this card is the map.

Two things it is trying to give you: a sense of which jobs the standard library already does well enough that a dependency is a mistake, and a sense of what each of the big third-party libraries is actually *for* — which is usually narrower than its README suggests.

The stdlib spine

These come with Python, work everywhere, and cover more of this course than you would expect. Every graded function you write in this course uses only these.

Module	What it is for here
<code>struct</code>	Fixed binary layouts. The single most-used module in the course.
<code>pathlib</code>	Paths, everywhere, instead of string concatenation.
<code>mmap</code>	Random access into an image too large to read into memory.
<code>hashlib</code>	Content hashing for identity, dedupe and provenance.
<code>binascii</code>	hex/base64 conversions at the byte level.
<code>ctypes</code>	Calling native code — and it is the <i>real</i> interop story, not a toy.
<code>socket</code>	Byte-level networking; the thing pwntools wraps.
<code>subprocess</code>	Driving another tool, which is most of P09.
<code>argparse</code>	The CLI half of every tool you build.
<code>itertools</code> / <code>collections</code>	The shapes in P05.
<code>re</code>	Text pattern matching — and note it works on <code>bytes</code> too.
<code>json</code>	The interchange format between your stages.

Reach for the stdlib first. A parser built on `struct` has no install story, no version skew, and runs inside a host application that will not let you pip install anything. That last point is the whole back half of this card.

The third-party field

Reading and disassembling

Capstone — *the disassembly library*.

A note on the name, because this platform overloads it. On TheRange, “capstone” also means a kind of task (`kind: capstone`) and a scoring model. The library is a completely unrelated thing that happens to share the word. Everywhere in this course, **Capstone** with a capital C is the disassembler.

Give it bytes and an architecture, get back instructions with mnemonics, operands and groups. It does not know about functions, sections or control flow — it decodes. That narrowness is why it is fast and why it is embedded in almost everything else. P08 builds a pass on it.

keystone — the mirror of Capstone: give it assembly text, get bytes. You need it far less often than you expect, and mostly when constructing a payload or a patch.

pyelftools — pure-Python ELF parsing, including DWARF. Read-only, and its DWARF support is the reason to pick it over the alternatives.

LIEF — parses PE, ELF and Mach-O *and writes them back out*. If you need to modify a binary and have it still load, this is the one. Heavier than pyelftools, and a C extension.

Driving, hooking and running

pwntools — the exploitation and CTF toolkit. What you will actually use most is the boring part: tubes (`remote` , `process`) that handle framing and buffering so you do not write another `recv` loop, and the packing helpers (`p64` , `u32`). P07 contrasts these with raw sockets so you know what they are doing for you.

Frida — dynamic instrumentation. Injects a JavaScript agent into a running process; your Python drives it and receives messages. The right tool when the question is “what does this program actually do at run time” and you have a running instance. P10.

unicorn — CPU emulation without an operating system. Run a function’s instructions with no process around them. **Qiling** builds an OS emulation layer on top of it.

angr — symbolic execution and binary analysis. Powerful, slow, and prone to state explosion; correct for narrow, well-posed questions.

Unicorn, Qiling and angr are taught properly in the **firmware course, module E4**, which is built. This course does not re-teach them.

r2pipe — drives radare2 as a subprocess and speaks JSON to it. The pattern generalises, and P09 is about that pattern.

Formats, patterns and protocols

construct — declarative binary structures. A nice middle ground between hand-rolled `struct` and a full parser library, and it handles length-dependent and conditional fields well.

yara-python — pattern matching over files, productised. Rules with wildcards, string sets and conditions. P08 has you build a small version of it first so the real one is not magic.

scapy — packet crafting and dissection, and pcap reading. Layer 2 and 3.

pycryptodome / **cryptography** — the two general crypto libraries. P06 uses them for recognising and undoing, not for breaking; the mathematics is the crypto course's.

Choosing: three questions

1. **Does the stdlib already do it?** Parsing a fixed header does not need a library. Reading DWARF does.
 2. **Do I need to write, or only read?** Reading ELF is `pyelftools`. Writing ELF is LIEF. Getting this backwards costs you a rewrite.
 3. **Where will this run?** The most important question, and the reason for the rest of this card.
-

The three interpreters

You will write Python for three different runtimes in this course, and they do not have the same rules. Assuming there is only one is a reliable way to write a script that works on your machine and fails inside the tool.

1. CPython 3.x — your own environment

The normal case. Anything on PyPI, a virtual environment per project, pinned versions.

```
python -m venv .venv && . .venv/bin/activate
pip install capstone lief
pip freeze > requirements.txt      # pin, or your tool rots
```

Use `pipx` for tools you want on your PATH globally rather than per-project.

2. Ghidra's bundled Jython — Python 2.7

Ghidra ships an embedded Jython interpreter so scripts can talk to its Java API directly. It is genuinely convenient and it is genuinely Python 2.7.

What that costs you:

- **Python 2.7 language semantics.** No f-strings. `print` behaves differently. Integer division differs. Text and bytes are not cleanly separated.
- **The Python 2.7 standard library only** — so any `stdlib` module that arrived in Python 3 simply is not there, no matter how ordinary it feels to you now.

- **No C extension modules at all.** Jython runs on the JVM; a library that compiles against CPython's C API cannot load. This rules out most of the interesting half of the list above.
- **No practical pip story.** Pure-Python packages can sometimes be made to work; assume nothing does.

The upshot: **a script that must run inside Ghidra has a dependency budget of approximately zero.** You write it against the Java API and the 2.7 standard library, and you keep it small. Anything heavier belongs on the host side.

Before you write a script for Ghidra, check every import against two questions: did this module exist in Python 2.7, and is it pure Python? If either answer is no, that import has to go — or the script has to move out of Ghidra.

3. Ghidrathon — CPython 3 inside Ghidra

Ghidrathon is an extension that hosts a real CPython 3 interpreter inside Ghidra, giving scripts the Java API *and* modern Python *and* your installed packages. It is the answer to almost every complaint in the previous section.

The cost is that it is an extension: it has to be installed and version-matched to your Ghidra, so a script that depends on it is a script other people cannot simply run. That is a real trade-off, not a formality.

The fourth option, and often the right one: do not run inside the tool at all. Drive it from outside with `subprocess` — Ghidra's `analyzeHeadless`, radare2 via `r2pipe`, IDA in batch mode — and keep your logic in an environment you control. P09 is that pattern in full.

Pinning, and why a tool nobody can install is not a tool

Every third-party library above has broken an API between minor versions at some point. Your portfolio repository should say exactly what it was built against:

```
capstone==5.0.1
lief==0.15.1
```

Not `capstone>=5`. A reviewer who clones your repository in a year and gets a stack trace will conclude your tool does not work, and they will be right in every way that matters.

Where each of these is picked up

- Capstone, keystone, yara-python → P08
- pyelftools, LIEF, construct → P04
- pwntools, scapy → P07
- Ghidrathon, ghidra_bridge, IDAPython, r2pipe → P09
- ctypes, cffi, Frida → P10
- Atheris, boofuzz → P11
- AFL++, QEMU mode, crash triage → *not here*. Firmware E1, and it is already built.

- **unicorn, Qiling, angr** → *not here*. Firmware **E4**, also built.

Module P02-bytes-and-struct

Bytes, struct, and the layout you were handed

Everything downstream of this module is a byte string with a layout. A PE section header, a SquashFS superblock, a UART frame, a savegame, a certificate — different subjects, one skill. Get fluent here and P04 through P11 are mostly application.

A format string is a layout. Read it as one.

Three ways to hold bytes

Type	Mutable	Slicing	Use it for
<code>bytes</code>	no	copies	what you read; the default
<code>bytearray</code>	yes	copies	building output, patching in place
<code>memoryview</code>	follows the source	no copy	walking a large image

The difference matters at scale and nowhere else. Slicing a 400 MB firmware image a thousand times to inspect a thousand records copies 400 MB a thousand times. A `memoryview` slice is a window:

```
mv = memoryview(blob)
record = mv[off:off + 19]          # no copy – a view onto the same memory
```

Two things to know before you reach for it. A `memoryview` **keeps the underlying object alive**, so holding views onto a giant buffer keeps the giant buffer resident. And `struct.unpack_from` takes a buffer directly, so most of the time you do not need the slice at all — which is the better fix.

Integers and bytes

```
int.from_bytes(b"\x00\x00\x04\x00", "little")    # 262144
int.from_bytes(b"\x00\x00\x04\x00", "big")       # 1024
(262144).to_bytes(4, "little")                   # b'\x00\x00\x04\x00'
```

The byte order argument is not optional and has no sensible default — if you find yourself unsure which to pass, that is the artifact's question, not Python's.

The one people miss: `signed=False` is the default.

```
int.from_bytes(b"\xff\xff\xff\xff", "little") # 4294967295
int.from_bytes(b"\xff\xff\xff\xff", "little", signed=True) # -1
```

Both are correct readings of the same four bytes. Which one is right depends on how the field was declared, and reading a signed length as unsigned is one of the most productive bug shapes there is — a small negative becomes an enormous positive somewhere downstream.

struct, and the prefix that decides everything

A format string is a sequence of field codes, optionally preceded by a byte-order prefix.

Code	C type	Bytes
<code>b / B</code>	signed / unsigned char	1
<code>h / H</code>	signed / unsigned short	2
<code>i / I</code>	signed / unsigned int	4
<code>q / Q</code>	signed / unsigned long long	8
<code>4s</code>	4 raw bytes	4
<code>x</code>	one pad byte, no value	1

Case is the signedness: lower-case signed, upper-case unsigned. That is the whole rule, and it is worth knowing cold because the two produce identical bytes and different numbers.

The prefix

Prefix	Byte order	Sizes	Alignment
<code>@ (default)</code>	native	native	native — inserts padding
<code>=</code>	native	standard	none
<code><</code>	little-endian	standard	none
<code>> / !</code>	big-endian	standard	none

@ is the default, and it is almost never what you want. You are parsing an artifact produced by some other machine, some other compiler, some other decade. Native anything is a property of the machine you happen to be sitting at. Say what the *format* is:

```
struct.unpack("<IHH", data) # say it
struct.unpack("IHH", data) # native order, native padding, and a bug waiting
```

The alignment trap

Under `@`, `struct` pads fields to their natural boundary exactly as a C compiler would. Under any explicit prefix, it does not.

```
struct.calcsize("@HI")    # 8 - two pad bytes so the I lands on a multiple of 4
struct.calcsize("<HI")    # 6 - no padding at all
```

Two bytes of difference on a two-field struct. On a real header the gap is larger, and the symptom is not an exception — it is every field after the first gap being read from the wrong offset, producing numbers that are wrong and plausible.

When a layout genuinely *does* have compiler padding — because it came from a C struct compiled with default alignment — do not switch to `@` and hope your machine agrees. Write the padding into the format explicitly with `x`, so the layout is stated rather than inherited.

calcsize and iter_unpack

```
FMT = "<4sIHBQ"
struct.calcsize(FMT)          # the record's size on disk
for rec in struct.iter_unpack(FMT, blob): # a whole array of them
    ...
```

`iter_unpack` requires the buffer to be an exact multiple of the record size and raises otherwise, which is a feature: a trailing partial record means you got the size wrong or the file is truncated, and both are things you want to hear about.

A worked record

The layout this module uses throughout — invented for the course, realistic in shape:

```
struct part_entry {
    char    magic[4]; /* "PTBL" */
    uint32_t offset;  /* byte offset of the partition */
    uint16_t flags;
    uint8_t  type;
    uint64_t size;    /* partition length in bytes */
};
```

As a format string, packed and little-endian: `<4sIHBQ`.

One entry, dumped:

```
00000000  50 54 42 4c 00 00 04 00  PTBL....
00000008  03 00 02 00 00 18 00 00  ....
00000010  00 00 00                ...
```

```
>>> struct.unpack("<4sIHBQ", rec)
(b'PTBL', 262144, 3, 2, 1572864)
```

Read the dump against the declaration until the correspondence is obvious — which four bytes are the magic, where the `offset` field starts, why `03 00` is three and not seven hundred and sixty-eight. That reading is the skill; the `unpack` call is the easy part.

Writing a hexdump

You will want one that does what you want rather than what `xxd` wants — with your field boundaries marked, your offsets based where your image is based:

```
def hexdump(blob: bytes, *, base: int = 0, width: int = 16) -> list[str]:
    out = []
    for i in range(0, len(blob), width):
        chunk = blob[i:i + width]
        hexpart = " ".join(f"{b:02x}" for b in chunk)
        text = "".join(chr(b) if 32 <= b < 127 else "." for b in chunk)
        out.append(f"{base + i:08x} {hexpart:<{width * 3 - 1}} {text}")
    return out
```

`32 <= b < 127` is the printable-ASCII range, and the `.` for everything else is convention rather than law. Note the function returns lines rather than printing them — P01's split, applied.

Bit fields

When several values share a byte, `struct` gets you the byte and you do the rest:

```
version = flags      & 0x0F      # low nibble
kind    = (flags >> 4) & 0x07      # next three bits
present = bool(flags & 0x80)      # top bit
```

Mask after shifting, and mask with exactly as many bits as the field has. A missing mask reads the neighbouring field's bits into yours and produces a value that is wrong in a way no exception will tell you about.

The CLI — `hexview`

The portfolio tool for this module: a struct-aware dumper that decodes records rather than only showing bytes. It imports `toolcore` from P01.

```
"""hexview - a hex dumper that knows the layout."""

import sys

from toolcore import build_parser, configure_logging, emit, EXIT_CLEAN

LAYOUTS = {
    "part_entry": [("magic", "4s"), ("offset", "I"), ("flags", "H"),
                  ("type", "B"), ("size", "Q")],
}

def main(argv=None):
    parser = build_parser("hexview", "struct-aware hex dumper")
    parser.add_argument("path")
    parser.add_argument("--layout", choices=sorted(LAYOUTS), default=None)
```

```

parser.add_argument("--base", type=lambda s: int(s, 0), default=0,
                    help="address the image is based at, e.g. 0x8000000")
parser.add_argument("--count", type=int, default=None,
                    help="stop after N records")
args = parser.parse_args(argv)
configure_logging(args)

blob = open(args.path, "rb").read()

if args.layout is None:
    for line in hexdump(blob, base=args.base):
        print(line)
    return EXIT_CLEAN

records = decode_records(blob, LAYOUTS[args.layout], base=args.base)
emit({"path": args.path, "records": records[:args.count]},
     as_json=args.json)
return EXIT_CLEAN

if __name__ == "__main__":
    sys.exit(main())

```

Add a layout by adding a row to `LAYOUTS`, which is the point — every artifact you meet in the next ten modules is a name and a list of field codes.

README.md

```

# hexview

Dump a binary, optionally decoding it as an array of fixed-layout records.

## Use
hexview FILE                                # plain dump
hexview FILE --layout part_entry --json      # decoded records
hexview FILE --base 0x80000000              # offsets as real addresses

## Layouts
Defined in `LAYOUTS` as (name, struct code) pairs, little-endian and packed.

## What it does not do
Variable-length and conditional fields. Everything here is a fixed layout;
anything else wants a real parser (see `construct`, or P04's `binid`).

```

What is graded

You assemble the wrapper. The trainer grades the function it calls:

```
decode_records(blob, layout, *, base=0) -> list[dict]
```

Given a blob and a layout as (name, code) pairs, it decodes every whole record it can and reports what it could not — which is the part with a right answer, and the part py-p2-hunt-short-buffer shows you the cost of getting wrong.

Module P03-files-and-streams

Files you cannot fit in memory, archives you cannot trust

Two problems live in this module, and they are less unrelated than they look.

A 4 GB image is not a read(). An archive from someone else is not a directory.

Both are about refusing to let the artifact decide what your program does.

Choosing how to read

Situation	Reach for
A few KB, you want all of it	<code>path.read_bytes()</code>
Large file, front to back, once	<code>open(...)</code> and iterate in chunks
Large file, jumping around	<code>seek</code> / <code>tell</code> , or <code>mmap</code>
Many random reads into one huge image	<code>mmap</code>
Testing a parser	<code>io.BytesIO</code> , so there is no file at all

`read_bytes()` on a 4 GB firmware image asks the allocator for 4 GB. It may succeed, which is worse than failing, because it will succeed on your machine and fail on the build server.

Streaming, and why hashing is the model case

```
def sha256_of(path, chunk=1 << 16):
    digest = hashlib.sha256()
    with open(path, "rb") as handle:
        while block := handle.read(chunk):
            digest.update(block)
    return digest.hexdigest()
```

Constant memory regardless of file size. The pattern generalises to anything that can consume a stream: a hash, a byte histogram, a magic check on the first bytes, a search for a fixed pattern. If your analysis can be expressed as “update some state with each chunk”, it never needs the whole file.

mmap, and when it is not the answer

```
with open(path, "rb") as handle:
    with mmap.mmap(handle.fileno(), 0, access=mmap.ACCESS_READ) as mm:
        if mm[0:4] == b"\x7fELF":
```

```
...
offset = mm.find(b"squashfs")
```

The operating system pages in what you touch, so random access into a large image costs only the pages you use — and `find` over a mapping is a fast memory search rather than a Python loop.

It is not free. A mapping consumes address space, it keeps the file open, and on a 32-bit target a large file may not map at all. If you are reading front to back once, plain chunked reads are simpler and just as fast.

Walking a tree, deterministically

```
for path in sorted(Path(root).rglob("*")):
    if not path.is_file() or path.is_symlink():
        continue
    rel = path.relative_to(root).as_posix()
```

Four decisions in four lines, and every one of them matters:

- **sorted**. `rglob` and `os.walk` return whatever order the filesystem gives, which differs between machines and can change when a file is rewritten. A manifest exists to be diffed; unsorted output cannot be.
- **is_file()**. Directories appear in `rglob` output too.
- **is_symlink()**. A symlink into `/etc` is not part of the corpus, and following it takes you somewhere you did not intend to look.
- **relative_to(...).as_posix()**. Absolute paths leak the machine you ran on. Backslashes make a Windows manifest differ from a Linux one for no reason. Relative-and-POSIX is what makes two manifests comparable.

Sort by the **relative path**, not by the filename. They differ as soon as there is a subdirectory, and sorting by name scatters directory structure through the output.

Entropy: a weak classifier, honestly used

Shannon entropy over the byte histogram, in bits per byte, from 0 to 8:

```
def entropy(blob):
    if not blob:
        return 0.0
    counts = Counter(blob)
    n = len(blob)
    return -sum((c / n) * math.log2(c / n) for c in counts.values())
```

Content	Roughly
One repeated byte	0.0
English text	4.0 – 4.5
Compiled code	5.5 – 6.5
Compressed or encrypted	7.9 – 8.0
Every byte value once	exactly 8.0

Two implementation traps, both in that function:

An empty file. Zero bytes has no histogram. Its entropy is 0.0 **by definition** — return it, do not compute it, or you will divide by zero.

Negative zero. For a file of one repeated byte, every term is $1.0 * \log_2(1.0)$, which is 0.0, and negating the sum gives -0.0 . It compares equal to 0.0 and serialises as -0.0 , so two manifests differ in a way nobody can explain. Normalise it.

And the limits, which matter more than the formula:

- **High entropy is not evidence of encryption.** Compressed data looks identical. So does a JPEG, a PNG and a zip.
- **It says nothing about a small file.** Sixteen distinct bytes score 4.0 and 256 distinct bytes score 8.0 purely because there are more of them.
- **It is an average.** A key hidden in an otherwise-plain file will not move the whole-file number. Sliding a window across the file and reporting the profile finds things the single figure hides.

Report it as a number with a band, never as a verdict.

Archives you did not create

Extracting an untrusted archive is one of the few places a triage tool can do real damage — not to the thing it is analysing, to the machine it is running on.

Three mechanisms, all of them ordinary archive features:

Traversal. A member named `rootfs/../../../../etc/cron.d/backdoor`. Joined to your destination and normalised, it lands outside it:

```
>>> posixpath.normpath(posixpath.join("/opt/extract",
...                                     "rootfs/../../../../etc/cron.d/backdoor"))
'/etc/cron.d/backdoor'
```

Absolute paths. A member named `/etc/hosts`. This one is worse than it looks, because the join does not fight you at all:

```
>>> posixpath.join("/opt/extract", "/etc/hosts")
'/etc/hosts'
```

`join` discards everything to the left of an absolute component. Your destination is not overridden by force; it is simply dropped.

Symlinks and links. A member that is a symlink to `/`, followed by a member that writes *through* it. Neither member escapes on its own.

The containment check

Resolve, then verify, then write:

```
def safe_destination(dest: Path, member_name: str) -> Path:
    target = (dest / member_name).resolve()
    if not target.is_relative_to(dest.resolve()):
        raise ValueError(f"{member_name!r} escapes {dest}")
    return target
```

`resolve()` is doing the work: it normalises `..`, and it follows symlinks that already exist — which is what catches the two-member symlink trick. A string comparison against `startswith` without resolving first is not a check, and `os.path.normpath` alone does not follow links.

Since Python 3.12 you should not be writing this at all. `tarfile` has an extraction filter, and `data` is the strict one:

```
tar.extractall(path=dest, filter="data")
```

It refuses absolute paths, traversal, links outside the destination, device nodes and setuid bits. PEP 706 made it the default in 3.14. Write the check by hand once so you know what the filter is doing, then use the filter — and know that `zipfile` has no equivalent, so for zips the check above is still yours.

The CLI — `scanfs`

```
"""scanfs – describe a corpus."""

import sys

from toolcore import build_parser, configure_logging, emit, EXIT_CLEAN, EXIT_FINDINGS

def main(argv=None):
    parser = build_parser("scanfs", "inventory a directory tree")
    parser.add_argument("root")
    parser.add_argument("--min-entropy", type=float, default=None,
                        help="only report files at or above this entropy")
    parser.add_argument("--duplicates-only", action="store_true")
    args = parser.parse_args(argv)
    configure_logging(args)

    manifest = build_manifest(args.root)  # <-- graded
```

```

    if args.min_entropy is not None:
        manifest["files"] = [f for f in manifest["files"]
                               if f["entropy"] >= args.min_entropy]

    if args.dupes_only:
        manifest["files"] = []

    emit(manifest, as_json=args.json)
    return EXIT_FINDINGS if manifest["duplicates"] else EXIT_CLEAN

if __name__ == "__main__":
    sys.exit(main())

```

README.md

scanfs

Inventory a directory tree: size, SHA-256, entropy, magic, and duplicate groups.

Use

```

scanfs ./firmware --json
scanfs ./firmware --min-entropy 7.9      # compressed or encrypted candidates
scanfs ./firmware --dupes-only

```

Determinism

Output is sorted by relative POSIX path. Two runs over the same tree produce identical output; two machines produce identical output.

What it does not do

It does not extract archives, and it does not decide that a high-entropy file is encrypted – compressed data is indistinguishable by this measure.

What is graded

```
build_manifest(root) -> dict
```

The walk, the hashes, the entropy, the magic, the duplicate groups, and an `errors` list that is honest about what it could not read. That is `py-p3-build-manifest-code`, graded against a real tree on disk.

Module P04-containers

Parsing a container you did not write

A container format is a map: a header saying what is inside, and a table saying where each piece lives. PE and ELF are the two you will meet most, and they are the same idea with different spelling.

Every offset in an untrusted header is an attacker-chosen integer. Parse like it.

That sentence is the module. Everything below is either how the maps are drawn or what goes wrong when you believe them.

The two specimens

This card ships two files. **Neither is a program** — the headers are complete and internally consistent, the section bodies are filler, and nothing executes. They were authored for this course by `build_assets.py`, which ships beside them and is byte-reproducible.

File	Format	Size
<code>specimen.exe</code>	PE32+, x86-64, three sections	9,728 bytes
<code>specimen.so</code>	ELF64, x86-64, <code>ET_DYN</code> , four program headers	2,048 bytes

The published SHA-256 of each is shown next to the download. **Check it.** Verifying a hash before you analyse something is not ceremony — it is how you know that the thing you are writing a finding about is the thing someone else will look at. Get in the habit here, where it costs nothing.

Dispatch on magic, never on the extension

```
def identify(blob: bytes) -> str:
    if blob[:2] == b"MZ":
        return "pe"  # maybe — see below
    if blob[:4] == b"\x7fELF":
        return "elf"
    return "unknown"
```

A file named `.exe` may be anything; a file named `.bin` is frequently a PE. The bytes are the only thing that is load-bearing.

MZ is necessary and not sufficient. It says DOS executable, which PE extends. The real check is one indirection further: read `e_lfanew` at offset `0x3c`, and confirm `PE\0\0` is at that offset.

```
e_lfanew = int.from_bytes(blob[0x3c:0x40], "little")
if blob[e_lfanew:e_lfanew + 4] != b"PE\0\0":
    ...  # MZ, but not a PE
```

Notice what just happened: **you read an offset out of the file and used it to index the file.** `e_lfanew` is attacker-chosen. If it is `0x7FFFFFFF`, that slice returns empty and compares unequal, which is fine — but the moment you use such a value for anything other than a comparison against a fixed literal, you need to have checked it. This is the first instance of the pattern and it recurs about six times per parser.

PE: the map

```
0x00 MZ ..... DOS header
0x3c e_lfanew -----
                        |
e_lfanew PE\0\0 <-----|
+4 COFF header (20 bytes)
    Machine, NumberOfSections, SizeOfOptionalHeader, Characteristics
+24 Optional header (240 bytes for PE32+)
    Magic 0x20b, ImageBase, SectionAlignment, FileAlignment,
    SizeOfImage, SizeOfHeaders, Subsystem, DllCharacteristics,
    then 16 data directories
+24+SizeOfOptionalHeader Section table: 40 bytes per section
```

Each section-table entry, in the order the fields appear:

Field	Size	Meaning
Name	8	NUL-padded, not NUL-terminated at 8 characters
VirtualSize	4	bytes it occupies once loaded
VirtualAddress	4	RVA — offset from ImageBase in memory
SizeOfRawData	4	bytes it occupies in the file
PointerToRawData	4	file offset
(12 bytes)		relocations and line numbers, obsolete
Characteristics	4	permissions and content flags

RVA to file offset

The conversion everything else depends on:

1. Find the section whose [VirtualAddress, VirtualAddress + VirtualSize) contains the RVA.
2. $\text{offset} = \text{rva} - \text{section.VirtualAddress} + \text{section.PointerToRawData}$

The delta is per section, and it is not a constant. Sections are aligned to SectionAlignment in memory and FileAlignment in the file — typically 0x1000 and 0x200 — so each one rounds up by a different amount and the gap between its memory and file positions drifts as you go down the table.

Worked, on a two-section image that is *not* the specimen:

section	virtual size	virtual address	raw size	raw offset	
.text	0x0800	0x1000	0x0A00	0x0400	delta 0xC00
.rdata	0x0400	0x2000	0x0200	0x0E00	delta 0x1200

An RVA of 0x2100 is in .rdata, so $0x2100 - 0x2000 + 0x0E00 = 0x0F00$. Computing 0xC00 from .text and subtracting it would give 0x1500, which is a perfectly plausible file offset

pointing at the wrong bytes. No error, no exception — just an answer that is wrong by 0x600 and looks exactly like an answer that is right.

The case with no answer. When `VirtualSize` greatly exceeds `SizeOfRawData`, the tail of that section is uninitialised data the loader zero-fills, and it does not exist in the file at all. An RVA past `SizeOfRawData` has no file offset. Report that; do not return a number.

ELF: the same idea, different spelling

```
0x00  7f 45 4c 46      e_ident
      [4] EI_CLASS  1 = 32-bit, 2 = 64-bit
      [5] EI_DATA   1 = little-endian, 2 = big
0x10  e_type        2 = ET_EXEC, 3 = ET_DYN
0x12  e_machine     62 = x86-64, 183 = AArch64, 40 = ARM, 8 = MIPS
0x20  e_phoff       -> program headers (what the LOADER uses)
0x28  e_shoff       -> section headers (what the LINKER used)
0x36  e_phentsize, e_phnum
```

Program headers are the ones that matter to you. Section headers can be stripped entirely and the binary still runs; program headers cannot. If a tool tells you a stripped ELF has no sections, that is true and irrelevant — the map you want is still there.

Mitigations: what `checksec` is actually reading

This is the part with no coverage anywhere else in this repository, and it is worth knowing at the byte level rather than as a tool's output.

PE — `DllCharacteristics`, a 16-bit field in the optional header

Bit	Name	Means
0x0020	HIGH_ENTROPY_VA	64-bit ASLR with full entropy
0x0040	DYNAMIC_BASE	ASLR at all
0x0080	FORCE_INTEGRITY	signature enforced
0x0100	NX_COMPAT	DEP / NX
0x0400	NO_SEH	no structured exception handlers
0x4000	GUARD_CF	Control Flow Guard

`DYNAMIC_BASE` without `HIGH_ENTROPY_VA` on a 64-bit image is a real finding: ASLR is on, but with 32-bit-era entropy. Two separate bits, routinely reported as one thing.

ELF — read from the program headers

Evidence	Means
<code>e_type == ET_DYN</code>	PIE — the image can be loaded anywhere
<code>PT_GNU_STACK</code> present, <code>PF_X</code> clear	NX on
<code>PT_GNU_STACK</code> present with <code>PF_X</code> set	executable stack — report it loudly
<code>PT_GNU_RELRO</code> present	RELRO, at least partial
<code>PT_GNU_RELRO</code> and <code>DT_BIND_NOW</code> in <code>.dynamic</code>	full RELRO

The partial-versus-full RELRO distinction needs **two independent pieces of evidence**, one from the program headers and one from the dynamic table. It is the single most commonly misreported item in this list, because the segment alone looks like the whole answer.

`ET_DYN` is also what a shared library looks like, so PIE and “is a library” are not distinguishable from `e_type` alone. `DT_FLAGS_1` carrying `DF_1_PIE` settles it when present, and often it is not. Say what you know.

Parse defensively, and report instead of raising

Four rules, in the order they save you:

- 1. Check before you read.** A field’s offset is only usable once you know the buffer extends that far. `struct.unpack_from` raises on a short buffer rather than improvising, which is why it is the right tool.
- 2. Bound every count.** `NumberOfSections` is two bytes, so it goes to 65,535. The section table is 40 bytes per entry at a known offset — so before you loop, check that `offset + 40 * n` fits inside the file. A count of 60,000 in an 8 KB file is not a section table; it is somebody testing you.
- 3. Accumulate errors; do not raise.** A tool that dies on the first malformed input cannot triage a corpus. Return what you learned plus a list of what you could not:

```
report = {"format": "pe", "sections": [...], "errors": []}
...
report["errors"].append("section 3: raw data extends past end of file")
```

That is P01’s exit-code contract with a bigger vocabulary: findings and errors are different things, and a partial parse reported as a complete one is the worst output available.

- 4. Never trust two fields to agree.** `SizeOfImage` and the section table’s extent are both in the file and can disagree. When they do, that is the finding.

When to stop hand-rolling

Hand-rolling is right for identification, triage and anything that must run where you cannot install packages. Reach for a library when:

You need	Use
DWARF, symbol tables, relocations on ELF	pyelftools
To modify a binary and keep it loadable	LIEF
Conditional or length-dependent field layouts	construct

`binid` stays hand-rolled, deliberately: it runs on damaged files that a strict parser refuses outright, and being able to report *why* something failed to parse is the whole product.

The CLI — `binid`

```
"""binid - identify a binary and report its build-time mitigations."""

import sys

from toolcore import build_parser, configure_logging, emit, summarize

def main(argv=None):
    parser = build_parser("binid", "identify a binary and report mitigations")
    parser.add_argument("paths", nargs="+")
    args = parser.parse_args(argv)
    configure_logging(args)

    results = []
    for path in sorted(args.paths):
        try:
            report = inspect_image(open(path, "rb").read()) # <-- graded
            findings = [{"id": f"missing-{m}", "severity": "medium"}
                        for m, on in report["mitigations"].items() if not on]
            results.append({"target": path, "findings": findings,
                           "error": report["errors"][0] if report["errors"]
                           else None})
        except Exception as exc: # noqa: BLE001
            results.append({"target": path, "findings": [], "error": str(exc)})

    report = summarize(results) # P01's function, reused unchanged
    emit(report, as_json=args.json)
    return report["exit_code"]

if __name__ == "__main__":
    sys.exit(main())
```

Note what this reuses. `build_parser`, `configure_logging`, `emit` and `summarize` all come from P01 untouched — the second tool cost you one function. That is the payoff for the split, and it is why P12 can compose eleven of these.

README.md

```
# binid

Identify PE and ELF images and report their build-time mitigations.

## Use
binid [--json] FILE [FILE ...]

## Reports
format · architecture · bits · endianness · entry point · sections
mitigations: nx · aslr · high_entropy_va · cfg · pie · relro

## Exit codes
0 clean · 1 missing mitigations reported · 2 something failed to parse

## What it does not do
Signature verification, packing detection, and imports. It also does not
guess: a field it could not read is reported in `errors`, never defaulted.
```

That last line is the one to mean. A parser that silently defaults an unreadable field to zero produces a report indistinguishable from a real one.

What is graded

```
inspect_image(blob) -> dict
```

Format, architecture, entry point, sections, mitigations, and an `errors` list that is honest about what it could not read. That is `py-p4-inspect-image-code`, and the two specimens above are what it is graded against.

Module P07-networking

`recv` does not return a message

One fact drives this whole module, and almost every protocol bug you will write comes from forgetting it:

`recv(n)` returns between 1 and `n` bytes. It has no idea what a message is.

TCP is a byte stream. The sender's `send()` calls and the receiver's `recv()` calls have no relationship to each other — the kernel, the network and the sender's buffering are all free to split and merge as they like. Three messages may arrive in one `recv`; one message may take six.

Everything below is how to live with that.

What actually goes wrong

Here is the code nearly everyone writes first:

```
data = sock.recv(4096)
n = int.from_bytes(data[:4], "little")
payload = data[4:4 + n]          # "the message"
```

It works on localhost, with small messages, on the first try. Then:

- **Two frames coalesce.** `recv` returns 15 bytes holding a 9-byte frame and a 6-byte frame. You read the first and **throw the second away**. It is not delayed, it is gone, and nothing raises.
- **One frame splits.** `recv` returns 6 bytes of a 9-byte frame. `data[4:4+n]` slices past the end, Python clamps it, and you get a short payload that looks like a real one.
- **The length prefix itself splits.** `recv` returns 2 bytes. You read a 4-byte integer from a 2-byte slice and get a completely fictional length.

None of these raises. All three produce plausible output. This is the same silent-short-slice failure as P02, arriving over a socket.

The fix: a residual buffer

Keep one buffer per connection. Append everything that arrives. Consume as many complete frames as the buffer holds. Keep the remainder for next time.

arrival	buffer	emitted
<code>recv → 05 00 00 00 41 4c</code>	<code>05 00 00 00 41 4c</code>	(nothing yet – payload incomplete)
<code>recv → 50 48 41 02 00 00 00 42 52</code>	<code>05 00 00 00 41 4c 50 48 41 02 00 00 00 42 52</code>	<code>b"ALPHA"</code> <code>b"BR"</code> residual: empty

Two rules make it correct, and both are checks that happen **before** a slice:

1. Is there a whole length prefix in the buffer? If not, stop and wait.
2. Does the buffer hold the payload that prefix declares? If not, stop and wait.

And one rule that is easy to miss: **after emitting a frame, loop again**. A single arrival can complete several frames, and a loop that emits one per arrival is the coalescing bug wearing a residual buffer as a disguise.

Three ways to frame

Style	Looks like	Bites you when
Length prefix	<code>[u32 len][payload]</code>	the prefix splits; the length is a lie
Delimiter	<code>payload\n</code>	the payload can contain the delimiter
Fixed record	every message is N bytes	the format grows a variant

Length-prefix is the common case in binary protocols and is what `probe` handles. Two things about it are worth stating plainly.

The length is attacker-controlled. It is a number that came out of the network. A four-byte prefix can say 4,294,967,295, and a loop that trusts it will happily wait forever for bytes that are never coming, or try to allocate four gigabytes. Cap it against something you chose:

```
MAX_FRAME = 1 << 20
if length > MAX_FRAME:
    raise ProtocolError(f"frame claims {length} bytes")
```

This is P04's bound-every-count rule, on a socket instead of a header.

Does the length include itself? Some protocols count the prefix in the length and some do not. Getting it wrong shifts you by exactly four bytes per frame, which desynchronises the stream from the second frame onward and produces garbage that looks like an encoding problem. Read the specification; if there is none, send two known frames and count.

The send side has the same problem

```
sock.send(payload)      # may send FEWER bytes and returns how many
sock.sendall(payload)   # loops until it is all gone, or raises
```

`send` returning a count is not a courtesy — it is the API telling you it may have done only part of the job. Use `sendall` unless you have a specific reason not to, and if you do use `send`, use its return value.

Sockets, concretely

```
import socket

with socket.create_connection(("10.0.0.7", 9001), timeout=5) as sock:
    sock.sendall(frame(b"PING"))
    buffer = b""
    while True:
        chunk = sock.recv(4096)
        if not chunk:          # b"" means the peer closed. NOT "no data yet".
            break
        buffer += chunk
    frames, buffer = frame_stream([buffer])
    for payload in frames:
        handle(payload)
```

Three things in there that are load-bearing:

- **timeout=5**. A socket with no timeout blocks forever, and a tool that hangs is worse than one that fails — it produces no output and no exit code.

- **if not chunk: break**. An empty `recv` means the peer closed the connection. It does not mean “nothing available right now”; that is a `BlockingIOError` or a timeout, and conflating the two produces a busy loop.
- **The buffer survives the iteration**. That is the entire point.

For several connections at once, reach for `selectors` — and give **each connection its own residual buffer**. One shared buffer across sockets interleaves two streams into nonsense, and it is a surprisingly common bug because it works perfectly until the second client connects.

pwntools tubes: what they do for you

```
from pwn import remote, p32, u32

io = remote("10.0.0.7", 9001)
io.send(p32(len(b"PING")) + b"PING")
length = u32(io.recv(4))      # exactly 4 bytes, or it waits
payload = io.recv(length)    # exactly `length` bytes, or it waits
```

`recv(n)` is the whole value proposition: **exactly n bytes or nothing**. It does the buffering and the looping that the residual buffer above does by hand. `recvuntil(delim)` is its delimiter-framed twin, and `recvline()` is `recvuntil(b"\n")`.

Also useful: `p32 / p64` and `u32 / u64` for packing and unpacking without writing `struct` format strings, and `context.endian = "big"` when the protocol is not little-endian.

What a tube does **not** do is know your protocol. It will hand you exactly the bytes you asked for, and if you ask for the wrong number — because the length included the prefix and you did not subtract it — it will hand you the wrong bytes without complaint, and every frame after that is misaligned. **Tubes fix the short-read problem. They do not fix the desynchronisation problem.**

Which is why this module has you write the loop first. Convenience you understand is a tool; convenience you do not is a place bugs hide.

Reading a capture instead of a socket

Sometimes the traffic already happened. `scapy` reads a pcap:

```
from scapy.all import rdpcap, TCP, Raw

payloads = [bytes(p[Raw].load) for p in rdpcap("session.pcap")
             if p.haslayer(TCP) and p.haslayer(Raw)]
frames, residual = frame_stream(payloads)
```

Note what that gets you for free: a pcap’s packet boundaries are *also* not message boundaries, so the same `frame_stream` handles both. A framing function that takes a list of chunks rather than a socket works on live traffic, on a capture, and in tests — which is why the graded function has that signature.

The CLI — probe

```
"""probe — a length-prefixed protocol client that cannot short-read."""

import socket
import struct
import sys

from toolcore import build_parser, configure_logging, emit, EXIT_CLEAN

def main(argv=None):
    parser = build_parser("probe", "speak a length-prefixed protocol")
    parser.add_argument("host")
    parser.add_argument("port", type=int)
    parser.add_argument("--header", default="<I", help="length prefix format")
    parser.add_argument("--send", action="append", default=[],
                        help="payload to send; repeatable")
    parser.add_argument("--timeout", type=float, default=5.0)
    args = parser.parse_args(argv)
    configure_logging(args)

    received, buffer = [], b""
    with socket.create_connection((args.host, args.port),
                                timeout=args.timeout) as sock:
        for text in args.send:
            body = text.encode()
            sock.sendall(struct.pack(args.header, len(body)) + body)
        while True:
            try:
                chunk = sock.recv(4096)
            except TimeoutError:
                break
            if not chunk:
                break
            buffer += chunk
            frames, buffer = frame_stream([buffer], args.header) # <-- graded
            received.extend(frames)

    emit({"host": args.host, "port": args.port,
         "frames": [f.hex() for f in received],
         "residual_bytes": len(buffer)}, as_json=args.json)
    return EXIT_CLEAN

if __name__ == "__main__":
    sys.exit(main())
```

`residual_bytes` is in the report on purpose. Leftover bytes at the end of a session mean the stream ended mid-frame — a truncated response, a protocol you have misread, or a length that included its own prefix. Reporting zero when it is not zero is the same lie P01 was about.

README.md

```
# probe

Speak a length-prefixed binary protocol, without short reads.

## Use
    probe HOST PORT --send PING --json
    probe HOST PORT --header '>I'      # big-endian length prefix

## Reports
frames received (hex) · residual_bytes

## What it does not do
TLS, delimiter or fixed-record framing, and it does not interpret payloads.
Non-zero `residual_bytes` means the stream ended mid-frame – that is a
finding, not a rounding error.
```

What is graded

```
frame_stream(chunks, header="<I") -> (frames, residual)
```

Given the chunks in the order they arrived, return every complete payload and whatever is left over. Chunk boundaries carry no information, several frames may share a chunk, one frame may span many, and a zero-length payload is still a frame. That is `py-p7-frame-stream-code`.

Module P08-static-analysis

Passes: scanning five thousand functions

Reading one function is reverse engineering. Writing the thing that reads five thousand and hands you the twelve worth looking at is tooling, and it is a different skill with different failure modes.

A pass turns judgement into a rule, applies it everywhere, and reports candidates — not verdicts.

That last clause is the discipline. A scanner that says “bug here” is wrong often enough to be useless. A scanner that says “these forty sites have an attacker-influenced length reaching a copy” is a tool you will use every week.

Capstone: bytes in, instructions out

Capstone — the disassembly library, not this platform’s `kind: capstone` — does exactly one thing: decode bytes into instructions. No container parsing, no function recovery, no control flow. That narrowness is why it is fast and why it is embedded in almost everything else.

```
from capstone import Cs, CS_ARCH_X86, CS_MODE_64
```

```
md = Cs(CS_ARCH_X86, CS_MODE_64)
for insn in md.disasm(code, 0x1000):
    print(f"{insn.address:#x} {insn.mnemonic} {insn.op_str}")
```

`disasm` is a generator, and it **stops silently at the first byte it cannot decode**. A pass that assumes it got the whole buffer will quietly analyse the first third of a function and report on it as though it were the whole. Compare what you consumed against what you passed in, or use `disasm_lite` and check.

Picking the architecture and mode

Target	Arch	Mode
x86-64	CS_ARCH_X86	CS_MODE_64
32-bit x86	CS_ARCH_X86	CS_MODE_32
AArch64	CS_ARCH_ARM64	CS_MODE_ARM
ARM32, A32	CS_ARCH_ARM	CS_MODE_ARM
ARM32, Thumb	CS_ARCH_ARM	CS_MODE_THUMB
MIPS, big-endian	CS_ARCH_MIPS	CS_MODE_32 + CS_MODE_BIG_ENDIAN

Two of these bite.

Thumb is a mode, not an architecture. The same ARM binary contains both A32 and Thumb code, and which one applies is a property of the *address* — the low bit of a function pointer carries it. Decode Thumb as A32 and you get valid, plausible, entirely wrong instructions. Not an error: a wrong answer.

MIPS is not reliably little-endian. Routers ship both. Get it wrong and, again, it decodes — into garbage that looks like code.

The pattern is worth naming because it recurs: **a wrong mode does not raise, it lies**. Sanity-check against something you know, such as whether the function prologues look like prologues.

`detail=True`, and the thing everyone hits

```
md = Cs(CS_ARCH_X86, CS_MODE_64)
md.detail = True # ← without this, operands is EMPTY

for insn in md.disasm(code, 0x1000):
    for op in insn.operands: # silently never runs otherwise
        ...
```

Without `detail`, Capstone decodes far less per instruction — it is a performance choice, and a sensible default for a tool that only wants mnemonics. `insn.operands` is then an empty list, `insn.regs_access()` is unavailable, and group membership is not populated.

A loop over an empty list is not an error. Your pass runs, finds nothing, reports zero findings, and exits 0. **This is the failure this module's error-hunt is about**, and it is worth meeting once here so you recognise it.

With detail on, useful things become available:

```
insn.group(CS_GRP_JUMP)          # is this any kind of branch?
insn.group(CS_GRP_CALL)
read, written = insn.regs_access()
op.type, op.reg, op.imm, op.mem.base, op.mem.disp
```

`regs_access()` is the one that pays for itself: “which instruction last wrote the register that ends up as the third argument” is a question you will ask constantly, and answering it by string-matching `op_str` is how you end up with a pass that fails on one addressing mode you did not think of.

keystone, the mirror

```
from keystone import Ks, KS_ARCH_X86, KS_MODE_64
ks = Ks(KS_ARCH_X86, KS_MODE_64)
encoding, count = ks.asm(b"mov rax, rdi; ret")
```

Assembly text in, bytes out. You need it less often than you expect — mostly when building a patch or a payload, or when you want to know what a sequence encodes to so you can go looking for it.

Where a disassembler lies

Two strategies, two failure modes, and you should know which one your tool used.

Linear sweep decodes from an address straight through, instruction after instruction. Simple and complete — and it happily decodes data as code. A jump table or an inline string in `.text` produces a run of nonsense instructions, and because x86 is a variable-length encoding, it may not resynchronise for several instructions after the data ends.

Recursive descent follows control flow from known entry points, decoding only what it can prove is reachable. It does not decode data as code — and it *misses* code it cannot prove reachable: indirect calls through a table, callbacks handed to a library, exception handlers, anything reached only through a computed target.

Neither is right. Linear sweep over-reports and recursive descent under-reports, and every real tool is a hybrid with heuristics. What matters for your pass is knowing which error yours makes, because that determines whether a site missing from your output means “not there” or “not proven reachable”.

Capstone itself is a pure decoder and does neither: you hand it a buffer and an address and it decodes forward. The strategy is whatever *you* wrap around it. That is a feature — you get to choose — and it means the choice is yours to get wrong.

Byte signatures

Sometimes you do not want instructions at all. You want to find a known sequence across a corpus, fast, without decoding anything.

This course's syntax is a subset of YARA's hex strings:

Token	Means
<code>48 89 e5</code>	these exact bytes
<code>??</code>	any single byte
<code>[4]</code>	skip exactly 4 bytes
<code>[0-16]</code>	skip between 0 and 16 bytes

So the standard x86-64 frame setup, tolerating any frame size:

```
55 48 89 e5 48 83 ec ??
```

`push rbp; mov rbp, rsp; sub rsp, <anything>`. The wildcard is doing real work — without it you would need one signature per frame size.

And a two-part signature, where the parts are near each other but not adjacent:

```
48 63 ?? [0-16] e8
```

A `movsxd` — a sign-extending 32-to-64 widening — followed within sixteen bytes by a call. That is a length being sign-extended and then passed to something, which is one of the more productive shapes in binary vulnerability research.

Jumps make a signature match variable-length, which forces a decision the syntax does not make for you: when several gaps would work, which match do you report? This course specifies the **shortest** — try the smallest gap first and report the first complete match. That is a choice, not a law; make it deliberately and write it down, because a tool whose match lengths depend on implementation detail cannot be diffed against its own previous output.

What real YARA adds and this course does not implement: nibble wildcards (`4?` matches any byte whose high nibble is 4), alternation (`(aa|bb)`), and the whole rule layer of named strings with boolean conditions over them. Build the core, then use the real tool — and know what it is doing, which is the point of building the core first.

A pass emits findings

The shape every pass in this course shares:

```
def find_candidates(functions) -> list[dict]:
    findings = []
    for fn in functions:
```

```

    for insn in disasm(fn):
        if not interesting(insn):
            continue
        findings.append({
            "id": "sign-extended-length",
            "severity": "medium",
            "target": fn.name,
            "evidence": {"address": insn.address, "insn": str(insn)},
        })
    return findings

```

It returns; it does not print. It emits a record per candidate with enough evidence for a human to check it in thirty seconds. And it says `candidate` in its vocabulary, not `vulnerability` — the tool narrows the field, the human decides.

That is the same `Finding` shape P01 defined, and the same discipline: `binid` reports missing mitigations, `sigscan` reports pattern hits, and neither claims to have found a bug.

The CLI — `sigscan`

```

"""sigscan - find byte signatures across a corpus."""

import pathlib
import sys

from toolcore import build_parser, configure_logging, emit, summarize

def main(argv=None):
    parser = build_parser("sigscan", "scan files for byte signatures")
    parser.add_argument("paths", nargs="+")
    parser.add_argument("-p", "--pattern", action="append", default=[],
                        help="hex signature; repeatable")
    parser.add_argument("--rules", type=pathlib.Path,
                        help="file of signatures, one per line")
    args = parser.parse_args(argv)
    configure_logging(args)

    patterns = list(args.pattern)
    if args.rules:
        patterns += [ln.split("#")[0].strip()
                     for ln in args.rules.read_text().splitlines()
                     if ln.split("#")[0].strip()]

    results = []
    for path in sorted(args.paths):
        try:
            hits = scan(pathlib.Path(path).read_bytes(), patterns) # <-- graded
            results.append({
                "target": path,
                "findings": [{"id": f"sig-{h['pattern']}", "severity": "low",
                             "offset": h["offset"], "length": h["length"]}

```

```

        for h in hits],
        "error": None,
    })
except Exception as exc:
    results.append({"target": path, "findings": [], "error": str(exc)}) # noqa: BLE001

report = summarize(results) # P01's function, still unchanged
emit(report, as_json=args.json)
return report["exit_code"]

if __name__ == "__main__":
    sys.exit(main())

```

Third tool, same spine. `build_parser`, `configure_logging`, `emit` and `summarize` have now been reused unchanged three times, which is the whole argument for the split P01 made.

README.md

```

# sigscan

Find byte signatures across a corpus. YARA-lite: `??` wildcards and `[n-m]`
bounded jumps.

## Use
sigscan FILE... -p '55 48 89 e5 48 83 ec ??'
sigscan FILE... --rules prologues.txt --json

## Match semantics
All matches, including overlapping. Jumps take the SHORTEST gap that completes
the match. Results are sorted by (offset, pattern).

## What it does not do
Nibble wildcards, alternation, and rule conditions – use YARA. It also does not
disassemble: a signature hit is a byte sequence, not an instruction.

```

What is graded

```
scan(blob, patterns) -> list[dict]
```

Every match of every pattern, as `{"pattern": index, "offset": n, "length": n}`, sorted by position then pattern. Wildcards, bounded jumps, shortest match, overlaps included. That is `py-p8-scan-code`.